

Software Engineering Design, Architecture & Professional Practice

SDLC, Design Patterns, Clean Code, Testing, Architecture & DevOps

Chapter 1: Software Development Life Cycle (SDLC)

The SDLC is the structured process a development team follows to plan, create, test, and deliver software. The choice of methodology profoundly affects team dynamics, delivery cadence, and product quality.

Waterfall	Sequential phases: Requirements -> Design -> Implementation -> Testing -> Deployment. Predictable but inflexible — changing requirements mid-project is costly. Best for well-defined, stable projects (embedded firmware, government contracts).
Agile (Scrum)	Iterative, 2-week sprints with working software delivered each cycle. Embraces changing requirements. Daily standups, sprint planning, retrospectives. Dominant methodology in product companies.
Kanban	Continuous flow model. Work items visualised on a board (To Do / In Progress / Done). WIP limits prevent overloading. Better for operations/support teams than feature development.
DevOps	Culture and practice that unifies Development and Operations. Automates testing, integration, and deployment to enable multiple releases per day safely.
Extreme Programming (XP)	Agile with engineering practices: TDD, pair programming, continuous integration, refactoring, simple design. Produces high-quality code but demands disciplined teams.

Chapter 2: Clean Code & SOLID Principles

Professional software engineering is not just about making code work — it is about writing code that other developers (including your future self) can read, understand, and confidently modify. Robert Martin's SOLID principles are the foundation of maintainable object-oriented design:

S — Single Responsibility	A class should have only one reason to change. Separate data access, business logic, and presentation into distinct classes.
O — Open/Closed	Open for extension, closed for modification. Add features via new subclasses or strategy injection — don't edit working code.
L — Liskov Substitution	Subclasses must be substitutable for their base class without breaking the program. A Square should NOT extend Rectangle if Square overrides setWidth.
I — Interface Segregation	No client should depend on methods it does not use. Split fat interfaces into smaller, focused ones.
D — Dependency Inversion	Depend on abstractions, not concretions. High-level modules should not import low-level modules directly — inject dependencies.

```
# Violates Single Responsibility – class does too many things
class UserManager:
    def create_user(self, data): ...
    def send_welcome_email(self, user): ... # <- email concern
    def save_to_database(self, user): ...   # <- persistence concern
    def generate_pdf_report(self): ...      # <- reporting concern

# Respects SOLID – each class has one job
class UserRepository:
    def save(self, user: User) -> None:
        self.db.insert('users', user.__dict__)

class EmailService:
    def send_welcome(self, user: User) -> None:
        self.mailer.send(to=user.email, template='welcome')

class UserService:
    def __init__(self, repo: UserRepository, email: EmailService):
        self.repo = repo      # Dependency Injection (D principle)
        self.email = email

    def register(self, data: dict) -> User:
        user = User(**data)
        self.repo.save(user)
        self.email.send_welcome(user)
        return user

# Easy to test – inject mock dependencies
class MockEmailService:
    def __init__(self): self.sent = []
    def send_welcome(self, user): self.sent.append(user.email)
```

Chapter 3: Design Patterns

Design patterns are proven, reusable solutions to commonly occurring problems in software design. They are not code — they are templates for thinking about structure. The Gang of Four (GoF) catalogue organises patterns into three categories:

Creational Patterns:

```
# Singleton – ensure only one instance exists (e.g. DB connection pool)
class DatabasePool:
    _instance = None
    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance._pool = cls._instance._create_pool()
        return cls._instance

# Factory Method – let subclasses decide which object to create
from abc import ABC, abstractmethod
class Notification(ABC):
    @abstractmethod
    def send(self, message: str): ...

class EmailNotification(Notification):
    def send(self, message): print(f"Email: {message}")

class SMSNotification(Notification):
    def send(self, message): print(f"SMS: {message}")

class NotificationFactory:
    @staticmethod
    def create(channel: str) -> Notification:
        return {'email': EmailNotification,
                'sms': SMSNotification}[channel]()

notifier = NotificationFactory.create('email')
notifier.send("Your order has shipped.")
```

Behavioural Patterns:

```

# Observer – notify multiple objects when state changes
class EventBus:
    def __init__(self):
        self._listeners: dict = {}

    def subscribe(self, event: str, handler):
        self._listeners.setdefault(event, []).append(handler)

    def publish(self, event: str, data=None):
        for handler in self._listeners.get(event, []):
            handler(data)

bus = EventBus()
bus.subscribe('order.placed', lambda d: print(f"Send email for order {d['id']}"))
bus.subscribe('order.placed', lambda d: print(f"Update inventory for {d['items']}"))
bus.publish('order.placed', {'id': 'ORD-42', 'items': ['Laptop', 'Mouse']})

# Strategy – swap algorithms at runtime
class SortContext:
    def __init__(self, strategy):
        self._strategy = strategy

    def sort(self, data: list) -> list:
        return self._strategy(data)

sorter = SortContext(strategy=sorted)      # use built-in Timsort
sorter.sort([5, 2, 8, 1])

```

Chapter 4: Testing — Quality Without Fear

A comprehensive test suite is the engineering equivalent of a safety net — it lets you refactor and add features confidently. The test pyramid defines the ideal distribution of test types:

Unit Tests (base of pyramid)	Test a single function or class in isolation. Fast (milliseconds), numerous (hundreds). Mock all external dependencies. Use pytest, JUnit, Jest.
Integration Tests (middle)	Test how multiple components interact — e.g., service layer + database. Slower, fewer. Use real (test) database instances.
End-to-End Tests (top)	Simulate real user journeys through the full stack. Slowest, fewest. Use Selenium, Playwright, Cypress.

```
# pytest – unit testing with Test-Driven Development (TDD)
import pytest
from decimal import Decimal
from app.services import OrderService, InsufficientStockError

class TestOrderService:
    def setup_method(self):
        self.repo = MockOrderRepository()
        self.inventory = MockInventoryService()
        self.service = OrderService(self.repo, self.inventory)

    def test_place_order_reduces_inventory(self):
        self.inventory.set_stock('SKU-001', 10)
        order = self.service.place_order(sku='SKU-001', qty=3)
        assert self.inventory.get_stock('SKU-001') == 7

    def test_place_order_raises_when_out_of_stock(self):
        self.inventory.set_stock('SKU-001', 2)
        with pytest.raises(InsufficientStockError):
            self.service.place_order(sku='SKU-001', qty=5)

    def test_order_total_calculated_correctly(self):
        self.inventory.set_price('SKU-001', Decimal('29.99'))
        order = self.service.place_order(sku='SKU-001', qty=3)
        assert order.total == Decimal('89.97')
```

Chapter 5: System Design & Architecture

System design is the process of defining the architecture, components, and data flow of a software system to meet functional and non-functional requirements — scalability, reliability, performance, and maintainability.

Core architectural patterns:

Layered (N-Tier)	Presentation -> Business Logic -> Data Access. Clear separation, easy to understand. Risk of tight coupling between layers.
Microservices	Decompose the application into small, independently deployable services communicating via APIs or message queues. High scalability and team autonomy. Operational complexity increases significantly.
Event-Driven	Services communicate asynchronously via events on a message broker (Kafka, RabbitMQ). Highly decoupled. Ideal for workflows where downstream steps don't need to complete synchronously.
CQRS + Event Sourcing	Command Query Responsibility Segregation: separate read and write models. Event Sourcing stores every state change as an immutable event log — complete audit trail and ability to replay history.

Hexagonal (Ports & Adapters)

Core domain logic is isolated from infrastructure (DB, HTTP, MQ) via interfaces. Makes the domain independently testable and infrastructure-swappable.

The CAP Theorem: In a distributed system, you can only guarantee two of the three properties simultaneously — Consistency (all nodes see the same data), Availability (every request gets a response), and Partition Tolerance (system works despite network failures). Since network partitions are inevitable, you must choose between CP (strong consistency, e.g. banking) or AP (high availability, e.g. social media feeds).